

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Zyvor Argus

Getting Started

2026-09-01

Wrong product? If you need multi-target Watchfloor SSO / RBAC / billing, that is **Argus Enterprise**, not this Mission Control path — see [Which product](#). This page is for **Community** Zyvor Argus (CLI + Mission Control).

Missing Docker / Node / Helm? Install companion packages first: [Install prerequisites](#).

What you need

Requirement	Notes
Python 3.11+ + Node 20+	<code>make install</code> wires CLI, npm, and Playwright Chromium
Target under test	Set <code>ZYVOR_BASE_URL</code> (e.g. <code>https://zyvor.dev</code>)
Optional LLM	<code>LLM_PROVIDER</code> + API key for NL create / richer generation
Optional GitHub	<code>gh auth login</code> + <code>ZYVOR_PRODUCT_REPO</code> for GitHub specs / PR comments

Full env table: [Admin basics](#) · repo `.env.example` .

1. Install

```
git clone https://github.com/zyvorai/zyvor-argus.git
cd Zyvor Argus
cp .env.example .env # set ZYVOR_BASE_URL at minimum
make install # argus CLI + Playwright browsers
```

2. First smoke (no LLM required)

```
argus-test-exec --grpc @smoke
```

3. Open Mission Control

```
argus serve --port 8080  
→ http://localhost:8080/dashboard
```

When `DASHBOARD_PASSWORD` is set, sign in at `/login` (defaults often `admin` / `Admin@321` on lab hosts — override via env).

Argus Enterprise (Watchfloor) uses Keycloak / OIDC instead. After a default install, smoke-test with `demo` / `demo` or `ssouser` / `Sso@321` — full guide: [Enterprise SSO / OIDC](#).

4. Orient yourself

1. **Overview hero** — pods, replicas, last QA run, pass rate, knowledge (Ask Zyra).
2. **Side rail** — Pipeline, Visual, Quality, Journeys, API, Probes, Security, Operations, **Ask Zyra**.
3. **Live job panel** — macOS Terminal log, ✓/✗ chips, Stop.
4. **Schedules / Findings / QA Runs** — Operations panel.
5. **⌘K / Search** — jump to any action; **theme toggle** for light mode.

5. First workflows

A. Smoke from the UI

Mission Control → ► Smoke (optional `@smoke` `grep`).

B. Drive a short journey (record video)

Actions → 🎬 Flow test — paste steps or English, run with video recording.

Against the public demo site:

```
ZYVOR_BASE_URL=https://zyvor.dev  
argus_flow_run https://zyvor.dev --steps /tmp/zyvor-home.steps --video
```

Full recipe + **watch the journey video**: [Test zyvor.dev](https://zyvor.dev).

C. Full pipeline from a spec

```
argus_test_run --source local --spec path/to/spec.mn
```

Related

- [Which product](#) — Community vs Watchfloor vs Enterprise v2 overlay
- [Using the Dashboard](#)
- [Test zyvor.dev](#)
- [Common workflows](#)
- [Admin basics](#)
- [Enterprise SSO / OIDC](#)
- [Page-by-page guides](#)

Operate from the console (UX)

1. Open this route from the nav or command palette and wait for live API data.
 2. Use filters/search when present; drill into a row for detail.
 3. For mutating actions: confirm role gates and impact before applying.
 4. **Empty / fail:** Check service health, auth, and that required CRDs/backends for this domain are installed.
 5. **Success:** Live data loads; created/updated objects appear without error toasts.
-

Zyvor Argus's **Mission Control** is a single live console at `/dashboard` (login at `/login` when password auth is on).

Layout

Element	Purpose
Side rail	Category navigation: Overview, Ask Zyra, Pipeline, Visual, Quality, Journeys, API, Probes, Security, Operations — collapsible to icons-only
Global header	Knowledge lamp, dark/light theme toggle, Search (opens ⌘K palette)
Overview hero	Health banner + stat tiles (pods, replicas, last run, pass rate, cron, knowledge)
Panel viewport	One active category at a time; hash URLs (<code>#pipeline</code> , <code>#ask</code> , ...) are bookmarkable
Workloads / Pods	K8s strip and pod cards with log drawer (needs kube SA or kubeconfig)
Action cards	Job cards inside each category panel — smoke, flow, HAR, probes, security, ...
Live job panel	macOS Terminal chrome — syntax-colored streaming log, case chips, Stop, copy/download log
Operations	Schedules, Findings, QA Runs, Videos
Command palette	⌘K / Ctrl-K or header Search — jump to any action or Ask Zyra

Theme

Dark charcoal theme is the default (Apple-style, minimal blue accent buttons). Use the **moon/sun** control in the header to switch to light mode; preference persists in the browser. Login uses the same design system.

Browse vs act

Reading pods, history, and findings is safe. Starting jobs mutates the target under test and writes reports under `reports/` — confirm URL, credentials, and scope before Run.

Practice on zyvor.dev

See [Test zyvor.dev \(with recording\)](#) for smoke + flow video + HAR against the public site.

Ask Zyra

Optional citation-first knowledge Q&A — side rail **Ask Zyra** or ⌘K. See [Tutorial 14](#).

Related

- [Getting Started](#)
- [Test zyvor.dev](#)
- [Page-by-page guides](#)
- [Common workflows](#)

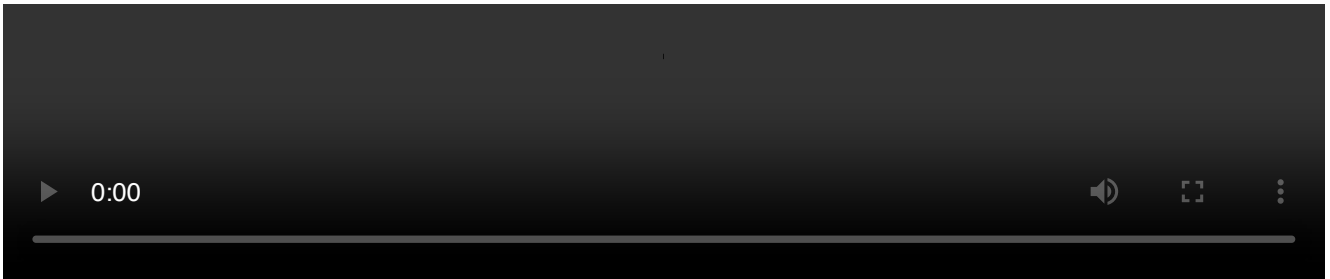
Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when auth is enabled).
 2. Pick a category from the side rail or #K.
 3. Fill the action card fields, run the job, watch the live terminal panel.
 4. Review Findings / QA Runs / downloads when the job completes.
 5. **Empty / fail:** Check `GET /health`, auth, and that Playwright browsers are installed on the host.
-

Canonical how-to for pointing Zyvor Argus at <https://zyvor.dev>. **Watch the journey video first**, then re-run the same flow locally.

Watch the recording

 Mission Control demo



- [Download the .webm](#)
- Steps used: `zyvor-dev-demo.steps` in `docs/assets/`
- Full tutorial: [Test zyvor.dev with recording](#)

Setup

```
zyvor.com
ZYVOR_BASE_URL=https://zyvor.dev
```

Re-run the same recording

```
argus test exec --grep @smoke
```

```
argus flow run https://zyvor.dev --steps docs/assets/zyvor-dev-demo.steps --video
```

Journey video lands under `reports/artifacts/flows/cli/journey.webm` (or Mission Control → 🎬 Videos).

Mission Control

```
argus serve --port 8080  
→ http://localhost:8080/dashboard
```

1. Pipeline → ► Smoke
2. Journeys → 🎬 Flow test — URL `https://zyvor.dev` , paste `zyvor-dev-demo.steps` , record video on
3. Journeys → 📹 HAR — optional record `/` then replay

UX cues

Cue	How
Side rail	Switch category panels (Pipeline, Journeys, ...)
Dark theme	Default; header moon/sun for light mode
⌘K / Search	Command palette
Live job panel	Terminal-style log while jobs run
Ask Zyra	Side rail → citation-first Q&A (optional)

Related

- [Getting Started](#)
- [Using the Dashboard](#)
- [Flow test](#)
- [Common workflows](#)

Operate from the console (UX)

1. Open this route from the nav or command palette and wait for live API data.
 2. Use filters/search when present; drill into a row for detail.
 3. For mutating actions: confirm role gates and impact before applying.
 4. **Empty / fail:** Check service health, auth, and that required CRDs/backends for this domain are installed.
 5. **Success:** Live data loads; created/updated objects appear without error toasts.
-

Smoke after a deploy

1. [Run tests](#) → ► Smoke (or `argus test exec --grep @smoke`)
2. Check [Findings](#) and [QA Runs](#)

Spec → tests → report

1. [Generate tests](#) or CLI `argus test run --source local --spec ...`
2. [Run tests](#) → Full pipeline
3. Optional: enable autofix via `ENABLE_AUTOFIX` (see Admin basics)

Record and replay a journey

1. [Flow test](#) — English or step DSL, record video
2. Or [Import codegen](#) from Playwright codegen
3. [HAR record / replay](#) when you need offline network fixtures

Public demo: [Test zyvor.dev](#) — watch the committed journey `.webm`, then re-run smoke / flow / HAR against `https://zyvor.dev`.

API, auth, and live data

1. [Auth & session](#) → save session file
2. [API contract](#) against OpenAPI
3. [Live data](#) for WebSocket / SSE
4. Reuse the session on [Flow test](#)

Visual confidence

1. [Route sweep](#) or [Visual regression](#)
2. [Compare URLs](#) for staging vs prod
3. [Web Vitals](#) for LCP / CLS / INP

Continuous monitoring

1. Configure the card once (e.g. audit URL or flow steps)
2. [Schedules](#) → pick kind + interval

Related

- [Getting Started](#)
- [Using the Dashboard](#)
- [Page index](#)

Operate from the console (UX)

1. Open this route from the nav or command palette and wait for live API data.
2. Use filters/search when present; drill into a row for detail.
3. For mutating actions: confirm role gates and impact before applying.
4. **Empty / fail:** Check service health, auth, and that required CRDs/backends for this domain are installed.
5. **Success:** Live data loads; created/updated objects appear without error toasts.